Southern New Hampshire University

Project Two: Summary and Reflection Report

Adalberto Flores

Southern New Hampshire University

CS-320

Rob Tuft

August 12, 2026

Southern New Hampshire University

Summary

For Project one, I developed and tested the contact, task and appointment services for the Grand Strand Systems mobile application. The project focused on backend services, so the work involved Java classes, in-memory storage, validation rules, and JUnit tests. There was no database, graphical user interface, or web API. Because of that, unit testing was the best approach for checking whether each object and service method followed the requirements.

My testing approach changed as I worked through the three features. At first, I mainly focused on making sure the Contact code worked with normal input and tested a few obvious invalid cases. The Contact implementation was mostly correct. The contact ID was immutable, the phone number used the required 10-digit validation, and the setters validated values before assigning them. However, feedback showed that my original Contact test suite did not fully prove every requirement. I had missed null tests for several fields, a phone number that was too long, a phone number with letters, setter validation tests, and some service error tests.

That feedback helped me improve how I approached the Task and Appointment services. Instead of thinking only about whether a test passed, I started using the requirements as a checklist. For every requirement, I asked what valid input should look like, what invalid input should be rejected, and what boundary value should be tested. For example, if a requirement said an ID could not be longer than 10 characters, I tested a valid ID with exactly 10 characters and an invalid ID with 11 characters. This helped make my later test suites more complete and more directly connected to the requirements.

The Appointment service shows that improvement. The `Appointment` class requires an appointment ID that is not null and no longer than 10 characters. It requires a date that is not null or in the past. It also requires a description that is not null and no longer than 50 characters. In AppointmentTest, I created a valid appointment and used the following assertions to verify each stored field:

```
assertEquals("1", appointment.getAppointmentID());
assertEquals(future, appointment.getAppointmentDate());
assertEquals("Dentist appointment", appointment.getDescription());
```

I also wrote separate tests for a null ID, an ID over 10 characters, a null date, a past date, a null description, and a description longer than 50 characters. The test `acceptsFieldsAtMaxLength()` checks that an ID with exactly 10 characters and a description with exactly 50 characters are accepted. The tests show that the application follows the limit in the requirements instead of only testing the normal case.

Another requirement was that the appointment ID could not be updated. The code supports this by declaring the field as `private final String appointmentID` and not including an ID setter. I tested that the date and description could change while the ID remained the same. I also used reflection to verify that the class did not contain a `setAppointmentID` method:

```
assertEquals("1", appointment.getAppointmentID());
assertThrows(NoSuchMethodException.class, () ->
    Appointment.class.getMethod("setAppointmentID", String.class));
```

This was more useful than assuming the ID was protected just because it was marked final. It gave me actual test evidence that the object design matched the requirement.

For the Appointment service, I tested adding, retrieving, and deleting appointments. I verified that one appointment could be added, multiple appointments could be stored under different IDs and a deleted appointment returned `null` when retrieved. I also added error-path tests. The service rejects a null appointment, duplicate appointment IDs null deletion IDs, and attempts to delete an appointment that does not exist. For example, this test confirms that the service does not silently accept an invalid deletion request

```
assertThrows(IllegalArgumentException.class,
    () -> service.deleteAppointment("999"));
```

Writing these tests was important because error paths were one of the major gaps in my earlier Contact work.

I used IntelliJ to measure statement coverage after receiving feedback that I needed to report an actual coverage percentage. My final Contact, Task, and Appointment tests reached 100% statement coverage. That result showed that every executable statement was run during the tests, including the validation and exception paths. Still, I understand that 100% statement coverage does not prove the program has no bugs. It does not guarantee that every combination of inputs, user action, or future integration issue has been tested. The coverage percentage was useful supporting evidence, but the stronger evidence came from connecting each requirement to valid, invalid, and boundary-value tests. Code coverage is helpful for finding code that did not run, but it should not replace thoughtful tests based on business requirements (Atlassian, n.d.)

I made the code technically sound by validating values before storing them. In the `Appointment` constructor and setters, invalid values cause an `IllegalArgumentException` before the object is changed. For example, `setAppointmentDate ()` rejects a null date or a date before the current time. My tests confirm that a valid future date can replace the old one while null and past dates are rejected. This matters because an appointment that was valid at creation should not be allowed to become invalid through an update.

The code also makes defensive copies of the `Date` object when saving and returning dates. This protects the appointment date from accidental outside changes because Java `Date` objects are mutable. The Appointment service also checks for null and duplicate IDs before storing an appointment in its `HashMap`. These checks make the service more reliable because it does not allow bad data or duplicate records into the collection.

I kept the test code efficient by avoiding repeated setup. In `AppointmentServiceTest`, I used a helper method:

```
private Appointment createSampleAppointment(String id) {
    Date futureDate = new Date(System.currentTimeMillis() + 60000);
    return new Appointment(id, futureDate, "Dentist appointment");
}
```

This reduced duplicated code and made the test setup consistent. Most tests followed a simple pattern: create the service or object. Perform one action and verify the result. The services also used in-memory `HashMap` storage, so tests ran quickly and did not require database setup or cleanup.

Southern New Hampshire University

Reflection

The main testing technique I used was JUnit unit testing. Unit testing focuses on small parts of the application, such as constructors, setters and service methods. For this project, unit testing let me test the Contact, Task, and Appointment objects separately from the service classes that manage them. It was appropriate because each milestone was an isolated backend service.

I also used black-box testing ideas, especially equivalence partitioning and boundary value analysis. Equivalence partitioning means grouping inputs that should have the same result. For example, future appointment dates are valid inputs, while past dates and null dates are invalid inputs. Boundary value analysis focuses on values at the edge of the requirement. My 10-character appointment ID and 50-character description tests are boundary tests because they check the maximum permitted values and values just beyond the limits. These techniques are useful in real software because invalid user input often happens at boundaries, such as a required field being empty or a value being one character too long.

I did not use integration testing, system testing, performance testing, security testing, or user acceptance testing because they were outside the scope of this project. Integration testing would become important if the services connected to a database or REST API. For example, unit tests can prove that `addAppointment()` stores an object in a `HashMap`, but an integration test would verify that an appointment saves and retrieves correctly through a real database connection. System testing would be needed once the mobile interface, backend services, database, and authentication system were

connected. Performance testing would matter if many users created or retrieved appointments at the same time. User acceptance testing would help confirm that the final application supports the customer's real scheduling needs.

I had to use caution because the classes were connected even though they were tested separately. The service assumes that every appointment added to the map has already passed the validation rules in the `Appointment` class. If I removed date validation from `setAppointmentDate ()`, an appointment could be valid when created but become invalid later when updated. A small change in one class could affect whether the service still meets the overall requirements. This is why I learned to test both the object behavior and the service behavior.

Bias was also a concern because I was testing my own code. It is easy to test only the inputs I expect users to provide because I already know what I intended the program to do. My early Contact tests showed this problem. I tested some length limits but missed null values, invalid phone formats, setter behavior, and service failure cases. I reduced that bias by using the requirements as a checklist, reading feedback closely, and intentionally trying to break the code with null values, duplicate IDs, missing IDs, past dates, and oversized strings.

Finally, this project showed me why discipline matters in software engineering. Cutting corners in testing can create technical debt even if the code looks correct at first. My early Contact implementation was functional, but the incomplete test suite meant important requirements were not proven. In a professional project that could allow a bug to reach production or make future changes risky. To avoid that, I plan to review

requirements before coding, write tests alongside the feature test both valid and invalid input, run coverage reports, and treat feedback as a way to improve my process. The biggest lesson from this project was that testing is not just about getting green check marks. It is about building evidence that the software does what it is supposed to do and rejects what it should not allow.

Southern New Hampshire University

References

Atlassian. (n.d.).*What is code coverage?* https://www.atlassian.com/continuous-delivery/software-testing/code-coverage